DATA 266 — Homework 2

NLP, RAG & Training Optimization

Ritika Neema

MS Applied Data Science, San José State University

September 2026

Student ID: 019306638 | Seed ID: 6638

Runtime: Google Colab — Tesla T4 GPU, PyTorch 2.11.0+cu128

Part 1 — Embedding-Based Transfer Learning (4 pts)

1.1 Approach

We loaded the pretrained word2vec-google-news-300 model (3 million words, 300 dimensions), extracted baseline nearest neighbors for five movie-review-relevant words, then fine-tuned the embeddings on 10,000 IMDB reviews. The fine-tuning procedure seeds a new Word2Vec model's vocabulary with the pretrained vectors and continues training for 5 epochs on the IMDB corpus. We then compare neighbors and measure how much each word's vector shifted.

1.2 Top-3 Nearest Neighbors — Before vs. After

Before fine-tuning (Google News embeddings):

Word	Neighbor 1	Neighbor 2	Neighbor 3
cast	casts (0.72)	casting (0.72)	Cast (0.66)
score	scoring (0.72)	scores (0.66)	scored (0.64)
plot	plots (0.76)	Plot (0.65)	plotting (0.63)
screen	screens (0.77)	onscreen (0.61)	LCD_screen (0.56)
review	reviewed (0.66)	reviewing (0.66)	reviews (0.64)

After fine-tuning on IMDB reviews:

Word	Neighbor 1	Neighbor 2	Neighbor 3
cast	actors (0.65)	casting (0.62)	performances (0.55)
score	soundtrack (0.65)	cinematography (0.58)	music (0.56)
plot	storyline (0.73)	story (0.72)	plots (0.60)
screen	screens (0.55)	stage (0.45)	capitalise (0.43)
review	comment (0.70)	reviews (0.67)	comments (0.62)

Key observation: after fine-tuning, neighbors shift from morphological variants (scoring, scored) toward movie-domain semantic neighbors (soundtrack, cinematography, storyline). This confirms domain-specific adaptation.

1.3 Embedding Shift — Cosine Similarity (Original vs. Fine-Tuned)

Word	Cosine Similarity	Shift Rank
review	0.5562	Most shifted
score	0.6344	2nd
screen	0.6358	3rd

plot	0.6883	4th
cast	0.7652	Least shifted

'review' shifted the most (0.556) — its meaning in movie reviews ("a critique/opinion") diverges significantly from its general-news usage ("a formal inspection/evaluation"). 'cast' shifted the least (0.765) — it already means "actors" in news contexts too, so the domain shift is smaller.

1.4 t-SNE Visualization

A t-SNE plot was generated for the word "score" showing its top-5 neighbors before and after fine-tuning projected into 2D. The before-cluster groups score with scoring/scores/scored (morphological), while the after-cluster groups score with soundtrack/cinematography/music (movie-domain semantic). The two clusters are spatially separated, visually confirming the embedding shift. (See notebook Section 1.7 for the plot.)

Part 2 — RAG Pipeline (4 pts)

2.1 Pipeline Architecture

We built a RAG pipeline from explicit LangChain components — no prebuilt RetrievalQA chain:

- Document Loader: wikipedia Python package → LangChain Document objects for 10 movie Wikipedia pages
- Text Splitter: RecursiveCharacterTextSplitter (chunk_size=500, overlap=50) → 1,360 chunks
- Embeddings: sentence-transformers/all-MiniLM-L6-v2 (local)
- Vector Store: FAISS
- Prompt Template: explicit PromptTemplate with context + question variables
- LLM: google/flan-t5-base loaded directly as AutoModelForSeq2SeqLM (250M params, local, no API key)

2.2 Q&A Results

Question	Answer	Top Chunk Source
Who directed Inception and what is its main plot idea?	Christopher Nolan	Inception (rank 1)
Who played the lead role in The Godfather?	Marlon Brando	The Godfather (rank 2)
What ship sank in the movie Titanic?	RMS Titanic	Titanic (rank 1)
Who is the main villain in The Dark Knight?	Joker	The Dark Knight (rank 2)
What award did Parasite win at the Oscars?	Academy Award for Best Picture	Parasite (rank 1)

2.3 Retrieval Success Rate

All 5 questions had the expected keyword present in at least one of the top-3 retrieved chunks.

Retrieval Success Rate: 5/5 = 100%

2.4 Chunk-Configuration Comparison

We re-ran Questions 1 and 2 with chunk_size=800, overlap=100 (producing 1,166 chunks instead of 1,360):

Question	Answer (500/50)	Answer (800/100)
Inception director + plot?	Christopher Nolan	Christopher Nolan
Godfather lead role?	Marlon Brando	Don Vito Corleone

The Godfather answer changed from the actor name to the character name — a chunk-configuration sensitivity failure analyzed below.

2.5 RAG Failure Analysis

Failure 1 — Incomplete LLM answer (Inception)

The retriever correctly returned the chunk containing "Christopher Nolan" and the dream-infiltration premise at rank 1. However, flan-t5-base only answered the first half of the compound question ("who directed") and dropped the plot description entirely. Root cause: the 250M-parameter model struggles with two-part questions when context is long (~1,500 chars). Fix: use a larger LLM or split compound questions.

Failure 2 — Chunk-config sensitivity (Godfather)

With 500/50 chunking, the answer was "Marlon Brando" (correct actor). With 800/100, the top chunk shifted to one that prominently mentions "Don Vito Corleone" (the character) while the actor's name appeared less saliently — causing the small LLM to latch onto the character name instead. Fix: use a larger LLM less prone to surface-level extraction, or add a re-ranking step.

Part 3 — Training Optimization Techniques (2 pts)

3.1 Experimental Setup

All experiments share the same configuration: a 3-layer MLP (512→1024→1024→10), synthetic random data, batch size 64, 50 training steps, Adam optimizer (lr=0.001), run on a Tesla T4 GPU. Only the one technique under test changes between runs.

3.2 Tensor Creation (CPU vs GPU)

We timed creation of 100 × (1024×1024) tensors three ways:

Method	Time (s)	Speedup vs CPU
CPU only	1.455	1.0×
GPU direct	0.008	180×
CPU → GPU transfer	1.241	1.2×

Takeaway: creating tensors directly on GPU avoids the PCIe bus transfer and is 180× faster. CPU→GPU transfer is actually slower than pure CPU allocation because it pays both the allocation cost and the bus copy.

3.3 Weight Initialization

Technique	Time (s)	Peak Memory (MB)	Final Loss
Default init	0.197	2298.71	2.3256
Xavier init	0.151	2298.71	2.3357

Both reach similar loss in 50 steps. Xavier's advantage becomes more pronounced in deeper networks where activation variance stability matters more.

3.4 Activation Checkpointing

Tested with a deeper model (8 hidden layers):

Technique	Time (s)	Peak Memory (MB)	Final Loss
No checkpointing	0.561	2438.85	2.2986
Activation checkpointing	1.155	2438.85	2.2986

Checkpointing adds ~2× wall-clock time (activations recomputed during backward pass) with identical loss. The memory savings are more visible with larger models and batch sizes; our model is small enough that the weight footprint dominates.

3.5 Gradient Accumulation

Technique	Time (s)	Peak Memory (MB)	Final Loss
Standard (batch=64)	0.294	2298.71	2.3256
Grad accum ×4 (micro=16)	0.905	2298.71	2.3622

Same effective batch size, similar final loss. The 3× wall-clock slowdown comes from 4× more forward/backward passes per step through the Python loop. The technique is valuable when GPU memory is the bottleneck — it simulates larger batches without increasing per-step memory.

3.6 Mixed Precision Training

Technique	Time (s)	Peak Memory (MB)	Final Loss
FP32 baseline	0.227	2298.71	2.3256
Mixed precision (FP16)	0.490	2298.72	2.3273

On this small model the FP16 overhead (autocast context manager + GradScaler bookkeeping) outweighs the tensor-core speedup. Mixed precision shines with larger models where the compute-bound matmuls dominate — the technique is standard in modern LLM training for exactly that reason.

3.7 Summary

All five techniques were tested under controlled conditions. The headline results:

- GPU-direct tensor creation is 180× faster than CPU — always allocate on the target device.
- Weight initialization (Xavier) helps convergence stability in deep networks.
- Activation checkpointing trades compute for memory — essential for large models.
- Gradient accumulation simulates larger batches without extra memory.
- Mixed precision (FP16) reduces memory and speeds up training at scale.